# Optimización del aprendizaje de los modelos seleccionados

Búsqueda de hiperparámetros, esquemas de validación cruzada y estrategias de ensamble

Guía explicativa del notebook **EP3_Optimizacion_Modelos.ipynb**

Machine Learning (MLY1101) · Duoc UC · Caso C — Spotify Tracks
Preparación para la Evaluación N.º 3

Alejandra González · Constanza González · Diego Villar
Docente: Marco Japke

Repositorio: github.com/ritnight/machine-learning-spotify

# Contenido

1. Resumen ejecutivo
2. Punto de partida: los modelos de la EP2
3. Conceptos: métodos, esquemas de validación y ensambles
4. Hiperparámetros: qué controla cada uno
5. Diseño experimental
6. Bloque A — CrossValidation: ¿qué esquema estima mejor?
7. Bloque B — LeaveOneOut
8. Bloque C — GridSearchCV vs. RandomizedSearchCV (Boosting)
9. Bloque D — Bagging vs. Boosting
10. Comparación de todas las combinaciones y elección
11. Evaluación final en prueba
12. Conclusiones y limitaciones
13. Guía del notebook y archivos generados
14. Preguntas frecuentes para la defensa

# 1. Resumen ejecutivo

El docente pidió probar distintos **métodos de optimización del aprendizaje** sobre los modelos seleccionados en la EP2: GridSearchCV, RandomizedSearchCV y validación cruzada; los esquemas KFold estratificado, KFold estándar y LeaveOneOut; y las estrategias de Bagging y Boosting. El objetivo es elegir la combinación que más mejora cada modelo, igual que se hizo al comparar algoritmos en la EP2.

El notebook nuevo prueba esas opciones sobre los dos modelos ganadores de la EP2, ambos **HistGradientBoosting**: uno de regresión (popularidad de 0 a 100) y uno de clasificación (bajo, medio o alto). Para que la comparación sea justa, todas las combinaciones se miden en un mismo conjunto de **validación** que ninguna búsqueda ve. Solo la combinación ganadora se evalúa en el conjunto de prueba.

| Tarea | Combinación elegida | Hiperparámetros | Resultado en prueba | EP2 |
|---|---|---|---|---|
| Regresión | Boosting + CrossValidation (sin búsqueda) + GroupKFold: se mantiene la EP2 | learning_rate 0,03 · max_leaf_nodes 63 · min_samples_leaf 20 · l2_regularization 0,5 · max_iter 600 | MAE 9,238 · RMSE 14,205 · $R^2$ 0,521 | Igual |
| Clasificación | Boosting + RandomizedSearchCV + GroupKFold | learning_rate 0,0224 · max_leaf_nodes 102 · min_samples_leaf 39 · l2_regularization 0,976 · max_iter 300 | ROC-AUC 0,911 · con umbral 0,28: recall "alto" 0,614, precisión "alto" 0,513, F1 macro 0,724 | AUC 0,910 · 0,608 · 0,501 · 0,719 |

**Hallazgos principales:**

- **GroupKFold es el esquema adecuado.** KFold y StratifiedKFold dejan reediciones de una misma canción a ambos lados del pliegue (12% de fuga) y subestiman el error: estiman un MAE de 9,15 cuando el real es 9,40.
- **Estratificar no cambia la estimación** con 57 mil canciones de ajuste.
- **LeaveOneOut es inviable** con todo el dataset (unos 6,6 días por configuración). En una submuestra de 500 canciones no estima mejor que KFold de 10 pliegues y cuesta unas 70 veces más.
- **Bagging no mejora al boosting** en ninguna tarea y cuesta entre 4 y 6 veces más.
- **RandomizedSearchCV ganó en clasificación** porque, con el mismo presupuesto, exploró más hiperparámetros. En regresión, la grilla solo confirmó los valores de la EP2.
- La mejora en clasificación es **pequeña** (+0,0006 de AUC en prueba), pero va en la misma dirección en validación y en prueba.

# 2. Punto de partida: los modelos de la EP2

En la EP2 se compararon cuatro algoritmos de regresión y cuatro de clasificación (estos últimos, con cinco técnicas de balanceo). En ambas tareas ganó **HistGradientBoosting**, y en clasificación, la versión **sin balanceo**, elegida por ROC-AUC. Sus hiperparámetros se buscaron con RandomizedSearchCV y GroupKFold de 3 pliegues. Los modelos entrenados están en la carpeta `models/`.

| Hiperparámetro | Regresión (EP2) | Clasificación (EP2) |
|---|---|---|
| learning_rate | 0,03 | 0,03 |
| max_iter | 600 | 300 |
| max_leaf_nodes | 63 | 63 |
| min_samples_leaf | 20 | 50 |
| l2_regularization | 0,5 | 0 |
| Parada temprana | Automática (más de 10.000 filas) | Automática (más de 10.000 filas) |
| Resultado en prueba (EP2) | MAE 9,24 · RMSE 14,21 · $R^2$ 0,52 | AUC 0,910 · F1 macro 0,713 (0,719 con umbral 0,28) |

El notebook nuevo carga esos modelos y lee sus hiperparámetros, para partir exactamente de la misma configuración. También reutiliza la base de modelamiento de la EP2 (una fila por canción, con sus géneros en columnas multi-hot), la partición de entrenamiento y prueba agrupada por artista + título, y el mismo preprocesamiento, incluido el target encoding del artista dentro del pipeline.

# 3. Conceptos: métodos, esquemas de validación y ensambles

## 3.1 Métodos de ajuste

| Método | Qué hace | Cuándo conviene | Costo |
|---|---|---|---|
| CrossValidation (cross_validate) | Evalúa una configuración fija en k pliegues: entrena con k−1 y mide en el restante. No busca hiperparámetros; estima el desempeño. | Para estimar cómo generaliza un modelo ya definido. | k ajustes |
| GridSearchCV | Prueba todas las combinaciones de una grilla de valores. Evalúa cada una con validación cruzada y elige la mejor. | Pocos hiperparámetros con valores candidatos claros. | combinaciones × k |
| RandomizedSearchCV | Sortea n configuraciones desde distribuciones (por ejemplo, uniformes) y evalúa cada una con validación cruzada. | Muchos hiperparámetros o rangos continuos: con el mismo costo explora más. | n × k |

## 3.2 Esquemas de validación cruzada

| Esquema | Cómo arma los pliegues | Riesgo en este dataset |
|---|---|---|
| KFold estándar | Reparte las canciones al azar en k grupos. | Ignora las reediciones: la misma canción (artista + título) puede quedar en entrenamiento y en validación. La estimación sale optimista. |
| StratifiedKFold | Como KFold, pero cada pliegue conserva la proporción de cada estrato. En clasificación, los estratos son las clases bajo, medio y alto; en regresión, los deciles de popularidad. | Tiene la misma fuga que KFold. |
| GroupKFold | Las canciones con la misma clave artista + título caen siempre en el mismo pliegue. | Sin fuga. Es el esquema de la EP2. |

| Esquema | Cómo arma los pliegues | Riesgo en este dataset |
|---|---|---|
| StratifiedGroupKFold | Agrupa como GroupKFold y además intenta estratificar. | Sin fuga. |
| LeaveOneOut (LOO) | n pliegues, cada uno con una sola canción de validación. | Requiere n ajustes por configuración (57.362 aquí, unos 6,6 días). Además, no agrupa reediciones. |

### 3.3 Estrategias de ensamble

| Estrategia | Idea | Qué reduce | Cómo se aplicó |
|---|---|---|---|
| Boosting | Entrena árboles en secuencia; cada uno corrige el error de los anteriores. | Sobre todo el sesgo | Es el modelo seleccionado (HistGradientBoosting). Se ajustan sus hiperparámetros. |
| Bagging | Entrena varias copias del modelo, cada una con una muestra bootstrap distinta (con reemplazo), y promedia sus predicciones o probabilidades. | Sobre todo la varianza | BaggingRegressor y BaggingClassifier con el HistGradientBoosting de la EP2 como modelo base. |

Random Forest, uno de los modelos comparados en la EP2, es un caso particular de bagging sobre árboles de decisión. Aquí se aplica bagging al modelo ya seleccionado para ver si promediar varias copias del boosting lo mejora.

## 4. Hiperparámetros: qué controla cada uno

| Hiperparámetro | Modelo | Qué controla | Efecto al subirlo |
|---|---|---|---|
| learning_rate | Boosting | Cuánto corrige cada árbol nuevo. | Aprende más rápido, con más riesgo de sobreajuste. Con valores bajos se necesitan más árboles. |
| max_iter | Boosting | Número máximo de árboles. La parada temprana detiene el entrenamiento antes si deja de mejorar. | Más capacidad y más tiempo. |
| max_leaf_nodes | Boosting | Número máximo de hojas por árbol (complejidad de cada árbol). | Captura más interacciones, con más riesgo de sobreajuste. |
| min_samples_leaf | Boosting | Mínimo de canciones en cada hoja. | Regulariza: hojas más estables y menos sensibles al ruido. |
| l2_regularization | Boosting | Penalización L2 sobre los valores de las hojas. | Suaviza las predicciones extremas y compensa los árboles grandes. |
| n_estimators | Bagging | Cantidad de copias del modelo base que se promedian. | Menos varianza, con rendimientos decrecientes; el costo crece en proporción. |
| max_samples | Bagging | Fracción de canciones de cada muestra bootstrap. | Cada copia ve más datos: menos sesgo y menos diversidad. |
| max_features | Bagging | Fracción de variables que ve cada copia. | Con valores bajos hay más diversidad, pero algunas copias pierden variables clave. |

## 5. Diseño experimental

### 5.1 Datos y particiones

| Conjunto | Canciones | Uso |
|---|---|---|
| Entrenamiento EP2 | 71.740 | Origen de todo el notebook (la misma partición de la EP2). |
| → Ajuste (80%) | 57.362 | Aquí corren todas las búsquedas y validaciones cruzadas. |
| → Validación (20%) | 14.378 | Vara común: cada combinación se reentrena con todo el ajuste y se mide una vez aquí. Ninguna búsqueda la ve. |

| Conjunto | Canciones | Uso |
|---|---|---|
| Prueba EP2 | 17.843 | Se usa una sola vez, con la combinación ganadora y el modelo de la EP2. |

La separación entre ajuste y validación se hace con `GroupShuffleSplit`, agrupando por artista + título: ninguna canción, ni sus reediciones, queda a ambos lados.

## 5.2 Por qué hace falta una validación común

Las estimaciones de validación cruzada **no son comparables entre esquemas**. KFold puede verse mejor que GroupKFold solo porque tiene fuga, no porque elija mejores hiperparámetros. Si se eligiera la combinación con mejor puntaje de validación cruzada, ganaría siempre el esquema con más fuga. Por eso cada combinación se reentrena con el conjunto de ajuste completo y se mide en la misma validación, que es independiente de todas.

## 5.3 Regla de selección (fijada antes de ver los resultados)

- Métrica principal, la misma de la EP2: **MAE** en regresión (menor es mejor) y **ROC-AUC macro** en clasificación (mayor es mejor), medidas en validación y redondeadas a 2 y 3 decimales.
- Primer desempate: RMSE (regresión) o F1 macro (clasificación).
- Segundo desempate: se prefiere el esquema sin fuga. Si dos esquemas eligieron la misma configuración, se queda el que estimó su desempeño sin fuga.
- Tercer desempate: el menor tiempo de cómputo.

## 5.4 Combinaciones evaluadas

| Bloque | Combinaciones | Para qué |
|---|---|---|
| A. CrossValidation | Modelo EP2 × {KFold, StratifiedKFold, GroupKFold, StratifiedGroupKFold}, 3 pliegues | Medir cuánto se equivoca cada esquema al estimar el desempeño real. |
| B. LeaveOneOut | Modelo EP2 en 500 canciones × {LOO, KFold 5, KFold 10, GroupKFold 5} | Comparar LOO con KFold donde LOO es factible. |
| C. Búsqueda (Boosting) | {GridSearchCV, RandomizedSearchCV} × {KFold, StratifiedKFold, GroupKFold} | Ver si el método y el esquema cambian los hiperparámetros elegidos y su desempeño real. |
| D. Bagging | {GridSearchCV, RandomizedSearchCV} × GroupKFold, con Bagging sobre el modelo EP2 | Ver si promediar varias copias mejora al boosting. |

En total, cada tarea compite con **9 combinaciones**: la referencia EP2 evaluada con CrossValidation, 6 de Boosting y 2 de Bagging. Los bloques A y B miden la calidad de los esquemas de estimación.

**Combinaciones excluidas y por qué:**

- **Búsqueda con LOO:** multiplicaría por 57.362 el costo de cada configuración.
- **Bagging con KFold o StratifiedKFold:** el efecto del esquema ya se mide en el bloque C. Repetirlo con Bagging, entre 5 y 10 veces más caro, no aporta información nueva, y esos esquemas tienen fuga.

## 5.5 Presupuesto y reproducibilidad

- Mismo presupuesto para Grid y Random: **8 configuraciones × 3 pliegues** en Boosting y **4 configuraciones × 3 pliegues** en Bagging (cada configuración de Bagging entrena entre 5 y 12 modelos).
- Semilla fija (42) en todas las particiones y búsquedas. RandomizedSearchCV sortea las mismas 8 configuraciones en cada esquema y en cada tarea: así, el esquema solo cambia el

puntaje, no las candidatas.

- El pipeline usa `memory` para no recalcular el preprocesamiento de un mismo pliegue en cada configuración.
- Cada experimento se guarda en `data/processed/ep3_cache/`. La primera ejecución tardó unas 2,5 horas en 4 núcleos; las siguientes reutilizan esos resultados y tardan cerca de 1 minuto.

# 6. Bloque A — CrossValidation: ¿qué esquema estima mejor?

Se evalúa el modelo de la EP2, con sus hiperparámetros fijos, mediante `cross_validate` en cuatro esquemas de 3 pliegues. Cada estimación se compara con el desempeño **real** del mismo modelo, entrenado con todo el ajuste y medido en validación. La **fuga** es el porcentaje de canciones de validación cuya clave artista + título también está en el entrenamiento del pliegue. El **optimismo** indica cuánto mejor se ve el modelo en la validación cruzada que en la realidad: un valor positivo significa que la validación cruzada promete más de lo que el modelo entrega.

## Regresión (MAE)

| Esquema | Fuga | MAE estimado | Desv. entre pliegues | MAE real (validación) | Optimismo | Tiempo |
|---|---|---|---|---|---|---|
| KFold (estándar) | 12,2% | 9,148 | 0,0547 | 9,404 | 0,256 | 28 s |
| StratifiedKFold (estratificado) | 12,1% | 9,179 | 0,0372 | 9,404 | 0,225 | 26 s |
| GroupKFold (agrupado) | 0,0% | 9,588 | 0,0650 | 9,404 | -0,185 | 26 s |
| StratifiedGroupKFold (estratificado y agrupado) | 0,0% | 9,798 | 0,0703 | 9,404 | -0,394 | 24 s |

## Clasificación (AUC)

| Esquema | Fuga | AUC estimado | Desv. entre pliegues | AUC real (validación) | Optimismo | Tiempo |
|---|---|---|---|---|---|---|
| KFold (estándar) | 12,2% | 0,9089 | 0,0018 | 0,9075 | 0,0014 | 37 s |
| StratifiedKFold (estratificado) | 12,0% | 0,9081 | 0,0012 | 0,9075 | 0,0006 | 42 s |
| GroupKFold (agrupado) | 0,0% | 0,9055 | 0,0008 | 0,9075 | -0,0020 | 40 s |
| StratifiedGroupKFold (estratificado y agrupado) | 0,0% | 0,9015 | 0,0018 | 0,9075 | -0,0060 | 39 s |

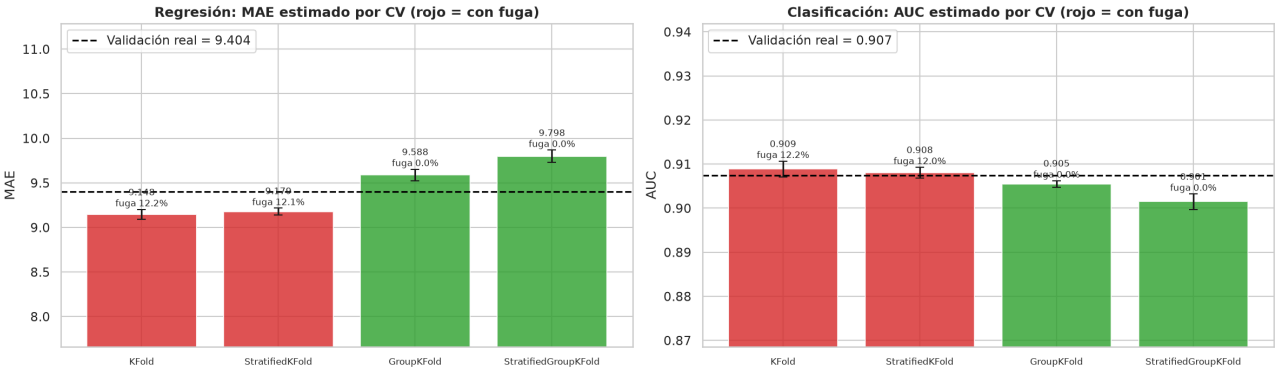



Figura 1. Estimación de cada esquema (barras) frente al desempeño real en validación (línea punteada). Rojo: esquemas con fuga.

### Interpretación

- **KFold y StratifiedKFold tienen un 12% de fuga.** Prometen un MAE de 9,15 y 9,18, pero el modelo obtiene 9,40: subestiman el error en unos 0,25 puntos. En AUC el efecto es menor (0,909 y 0,908 frente a 0,907), porque el AUC mide la capacidad de ordenar y no el tamaño del error.
- **Los esquemas agrupados son conservadores** (MAE de 9,59 y 9,80). Es esperable: cada pliegue entrena con dos tercios del ajuste y el target encoding del artista ve menos canciones.

Para decidir, es preferible un error sobreestimado que uno subestimado.

- **Estratificar casi no cambia la estimación**, aunque reduce algo la variación entre pliegues en regresión (0,037 frente a 0,055). Con tantas canciones, los pliegues aleatorios ya quedan balanceados.
- **Conclusión:** el esquema adecuado para este dataset es GroupKFold.

# 7. Bloque B — LeaveOneOut

LeaveOneOut entrena el modelo n veces, dejando fuera una canción cada vez. Con las 57.362 canciones de ajuste serían 57.362 ajustes por configuración, unos 6,6 días a ~10 segundos por ajuste. Por eso se toma una **submuestra aleatoria de 500 canciones** y se compara LOO con KFold de 5 y 10 pliegues y con GroupKFold de 5. Como cada pliegue de LOO tiene una sola canción, las métricas se calculan sobre todas las predicciones fuera de pliegue (`cross_val_predict`) en los cuatro esquemas. El valor "real" es el de un modelo entrenado con esas 500 canciones y medido en las 14.378 canciones de validación.

## Regresión

| Esquema | Ajustes del modelo | MAE | RMSE | Tiempo |
|---|---|---|---|---|
| LeaveOneOut | 500 | 17,470 | 21,936 | 288,2 s |
| KFold 5 | 5 | 17,909 | 22,086 | 3,8 s |
| KFold 10 | 10 | 17,575 | 21,977 | 6,2 s |
| GroupKFold 5 | 5 | 18,048 | 22,641 | 3,8 s |
| Real (500 canciones → validación) | 1 | 17,252 | 21,765 | 1,6 s |

## Clasificación

| Esquema | Ajustes del modelo | AUC | F1_macro | Tiempo |
|---|---|---|---|---|
| LeaveOneOut | 500 | 0,597 | 0,352 | 221,2 s |
| KFold 5 | 5 | 0,599 | 0,367 | 2,7 s |
| KFold 10 | 10 | 0,609 | 0,363 | 4,8 s |
| GroupKFold 5 | 5 | 0,592 | 0,367 | 2,6 s |
| Real (500 canciones → validación) | 1 | 0,606 | 0,401 | 1,2 s |

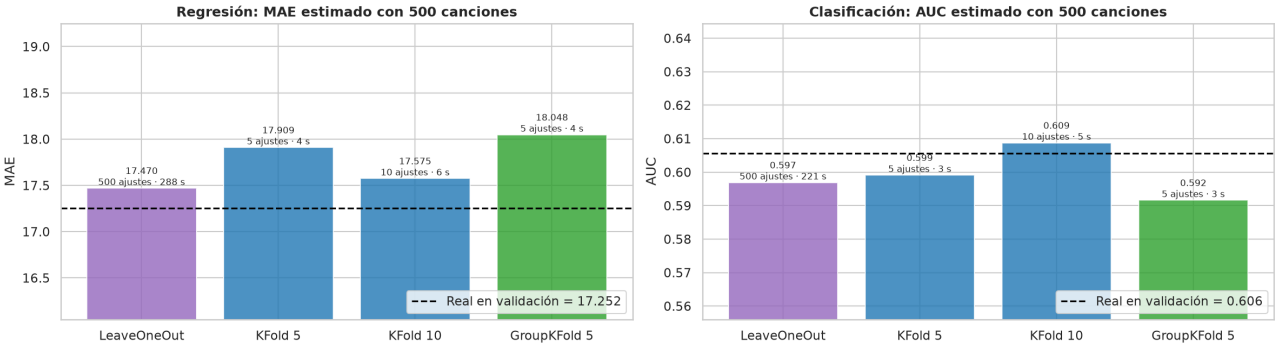



Figura 2. Estimaciones con 500 canciones y su costo. Línea punteada: desempeño real en validación.

### Interpretación

- Con 500 canciones el modelo es débil (MAE ≈ 17, al nivel del baseline; AUC ≈ 0,60). Este bloque compara **métodos de estimación**, no la calidad del modelo.
- En regresión, LOO queda más cerca del valor real (17,47 frente a 17,25), como predice la teoría: entrena con 499 de 500 canciones, casi sin sesgo. Pero KFold 10 queda muy cerca (17,58).
- En clasificación, LOO no es mejor (0,597 frente a 0,606 real; KFold 10 estima 0,609). Con una sola canción por pliegue, su estimación es muy variable.

- **Costo:** LOO necesitó 500 ajustes (288 s y 221 s), frente a 5 ajustes y unos 4 s de KFold 5: unas 70 veces más. Tampoco agrupa reediciones.
- **Conclusión:** LOO no es adecuado para este caso. KFold agrupado de 5 a 10 pliegues da estimaciones equivalentes por una fracción del costo.

# 8. Bloque C — GridSearchCV vs. RandomizedSearchCV (Boosting)

## 8.1 Espacios de búsqueda

| Hiperparámetro | GridSearchCV (2 × 2 × 2 = 8) | RandomizedSearchCV (8 sorteos) |
|---|---|---|
| learning_rate | {0,03; 0,06} | log-uniforme en [0,02; 0,15] |
| max_leaf_nodes | {31; 63} | entero en [15; 127] |
| min_samples_leaf | {20; 50} | entero en [10; 100] |
| l2_regularization | Fijo en el valor EP2 (0,5 en regresión; 0 en clasificación) | uniforme en [0; 1,5] |
| max_iter | Fijo en el valor EP2 (600 / 300) | Fijo en el valor EP2 (600 / 300) |

La grilla prueba combinaciones exactas de pocos valores alrededor de los de la EP2. La búsqueda aleatoria, con el mismo costo, explora además `l2_regularization` y valores intermedios. Cada búsqueda se corrió con KFold, StratifiedKFold y GroupKFold.

## 8.2 Hiperparámetros elegidos y desempeño

### Regresión

| Método | Esquema | Hiperparámetros elegidos | MAE CV | MAE validación | RMSE validación | Tiempo |
|---|---|---|---|---|---|---|
| GridSearchCV | GroupKFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 9,588 | 9,404 | 14,377 | 173 s |
| GridSearchCV | KFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 9,148 | 9,404 | 14,377 | 172 s |
| GridSearchCV | StratifiedKFold | learning_rate=0.06, max_leaf_nodes=63, min_samples_leaf=20 | 9,165 | 9,399 | 14,435 | 168 s |
| RandomizedSearchCV | GroupKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,619 | 9,416 | 14,392 | 199 s |
| RandomizedSearchCV | KFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,219 | 9,416 | 14,392 | 184 s |
| RandomizedSearchCV | StratifiedKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,210 | 9,416 | 14,392 | 189 s |

### Clasificación

| Método | Esquema | Hiperparámetros elegidos | AUC CV | AUC validación | F1_macro validación | Tiempo |
|---|---|---|---|---|---|---|
| GridSearchCV | GroupKFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9062 | 0,9081 | 0,7084 | 277 s |
| GridSearchCV | KFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9096 | 0,9081 | 0,7084 | 268 s |

| Método | Esquema | Hiperparámetros elegidos | AUC CV | AUC validación | F1_macro validación | Tiempo |
|---|---|---|---|---|---|---|
| GridSearchCV | StratifiedKFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9085 | 0,9081 | 0,7084 | 272 s |
| RandomizedSearchCV | GroupKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9057 | 0,9083 | 0,7088 | 278 s |
| RandomizedSearchCV | KFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9094 | 0,9083 | 0,7088 | 269 s |
| RandomizedSearchCV | StratifiedKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9084 | 0,9083 | 0,7088 | 282 s |

## 8.3 Todas las configuraciones probadas (con GroupKFold)

**Regresión · GridSearchCV** (ordenadas de mejor a peor)

| Hiperparámetros | MAE CV | RMSE CV | Seg. por ajuste |
|---|---|---|---|
| learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 9,5884 | 14,7291 | 8,3 |
| learning_rate=0.06, max_leaf_nodes=63, min_samples_leaf=20 | 9,6062 | 14,7499 | 4,5 |
| learning_rate=0.06, max_leaf_nodes=31, min_samples_leaf=20 | 9,6339 | 14,7517 | 5,4 |
| learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=50 | 9,6699 | 14,7359 | 8,2 |
| learning_rate=0.03, max_leaf_nodes=31, min_samples_leaf=20 | 9,6874 | 14,7164 | 7,1 |
| learning_rate=0.06, max_leaf_nodes=63, min_samples_leaf=50 | 9,6974 | 14,7620 | 5,5 |
| learning_rate=0.03, max_leaf_nodes=31, min_samples_leaf=50 | 9,7197 | 14,7304 | 6,4 |
| learning_rate=0.06, max_leaf_nodes=31, min_samples_leaf=50 | 9,7236 | 14,7806 | 5,6 |

**Regresión · RandomizedSearchCV** (ordenadas de mejor a peor)

| Hiperparámetros | MAE CV | RMSE CV | Seg. por ajuste |
|---|---|---|---|
| l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,6192 | 14,7201 | 11,4 |
| l2_regularization=0.319, learning_rate=0.0288, max_leaf_nodes=35, min_samples_leaf=42 | 9,7120 | 14,7393 | 6,7 |
| l2_regularization=0.0871, learning_rate=0.115, max_leaf_nodes=114, min_samples_leaf=33 | 9,7313 | 14,8293 | 3,7 |
| l2_regularization=0.55, learning_rate=0.0501, max_leaf_nodes=61, min_samples_leaf=71 | 9,7768 | 14,8228 | 7,2 |
| l2_regularization=0.437, learning_rate=0.0686, max_leaf_nodes=56, min_samples_leaf=69 | 9,7890 | 14,8120 | 5,4 |
| l2_regularization=0.898, learning_rate=0.0274, max_leaf_nodes=97, min_samples_leaf=96 | 9,7964 | 14,8094 | 11,9 |
| l2_regularization=0.456, learning_rate=0.0576, max_leaf_nodes=122, min_samples_leaf=98 | 9,8928 | 14,8880 | 6,8 |
| l2_regularization=0.562, learning_rate=0.136, max_leaf_nodes=121, min_samples_leaf=81 | 9,9923 | 15,0085 | 4,8 |

**Clasificación · GridSearchCV** (ordenadas de mejor a peor)

| Hiperparámetros | AUC CV | F1_macro CV | Seg. por ajuste |
|---|---|---|---|
| learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9062 | 0,7075 | 12,7 |
| learning_rate=0.06, max_leaf_nodes=31, min_samples_leaf=20 | 0,9060 | 0,7069 | 7,9 |
| learning_rate=0.03, max_leaf_nodes=31, min_samples_leaf=20 | 0,9059 | 0,7059 | 10,8 |
| learning_rate=0.06, max_leaf_nodes=63, min_samples_leaf=20 | 0,9056 | 0,7072 | 7,8 |
| learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=50 | 0,9055 | 0,7045 | 13,0 |
| learning_rate=0.03, max_leaf_nodes=31, min_samples_leaf=50 | 0,9053 | 0,7040 | 9,9 |
| learning_rate=0.06, max_leaf_nodes=31, min_samples_leaf=50 | 0,9052 | 0,7058 | 7,4 |
| learning_rate=0.06, max_leaf_nodes=63, min_samples_leaf=50 | 0,9049 | 0,7058 | 7,6 |

**Clasificación · RandomizedSearchCV** (ordenadas de mejor a peor)

| Hiperparámetros | AUC CV | F1_macro CV | Seg. por ajuste |
|---|---|---|---|
| l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9057 | 0,7092 | 17,6 |
| l2_regularization=0.319, learning_rate=0.0288, max_leaf_nodes=35, min_samples_leaf=42 | 0,9057 | 0,7062 | 10,4 |
| l2_regularization=0.437, learning_rate=0.0686, max_leaf_nodes=56, min_samples_leaf=69 | 0,9046 | 0,7080 | 6,2 |
| l2_regularization=0.55, learning_rate=0.0501, max_leaf_nodes=61, min_samples_leaf=71 | 0,9046 | 0,7060 | 8,6 |
| l2_regularization=0.898, learning_rate=0.0274, max_leaf_nodes=97, min_samples_leaf=96 | 0,9037 | 0,7067 | 16,1 |
| l2_regularization=0.0871, learning_rate=0.115, max_leaf_nodes=114, min_samples_leaf=33 | 0,9029 | 0,7051 | 5,4 |
| l2_regularization=0.456, learning_rate=0.0576, max_leaf_nodes=122, min_samples_leaf=98 | 0,9029 | 0,7035 | 7,7 |
| l2_regularization=0.562, learning_rate=0.136, max_leaf_nodes=121, min_samples_leaf=81 | 0,9009 | 0,7011 | 4,8 |

### Interpretación

- **Regresión: ninguna búsqueda mejora a la EP2.** La mejor configuración de la grilla coincide con los hiperparámetros de la EP2 (MAE 9,404 en validación). La variante con learning_rate 0,06, que eligió StratifiedKFold, baja el MAE en 0,005 pero empeora el RMSE (14,44 frente a 14,38). La configuración de la búsqueda aleatoria obtiene un MAE de 9,416.
- **Clasificación: ambas búsquedas mejoran levemente.** El AUC de validación pasa de 0,9075 (EP2) a 0,9081 con la grilla (min_samples_leaf 50 → 20) y a 0,9083 con la búsqueda aleatoria: árboles más grandes (102 hojas), compensados con regularización L2 de 0,976 y una tasa de aprendizaje menor (0,0224).
- **El esquema cambió la estimación, no la elección.** En 5 de los 6 casos, los tres esquemas eligieron la misma configuración. Las estimaciones con fuga siguen siendo optimistas: la grilla con KFold estima un MAE de 9,15 y esa misma configuración obtiene 9,40 en validación.
- **Grilla frente a búsqueda aleatoria:** con el mismo costo, entre 170 y 280 s por búsqueda, la aleatoria encontró la mejor configuración en clasificación, porque exploró dimensiones que la grilla dejaba fijas.

# 9. Bloque D — Bagging vs. Boosting

Se envuelve el HistGradientBoosting de la EP2, con sus hiperparámetros, en un `Bagging`: varias copias entrenadas con muestras bootstrap distintas, cuyas predicciones o probabilidades se promedian. Se usa GroupKFold, el esquema sin fuga, con ambos métodos de búsqueda.

| Hiperparámetro | GridSearchCV (2 × 2 = 4) | RandomizedSearchCV (4 sorteos) |
|---|---|---|
| n_estimators | {5; 10} | entero en [5; 12] |
| max_samples | {0,5; 0,8} | uniforme en [0,4; 0,9] |
| max_features | 1,0 (todas las variables) | uniforme en [0,7; 1,0] |
| bootstrap | Sí (muestreo con reemplazo) | Sí |

### Regresión — mejor configuración de cada búsqueda

| Método | Hiperparámetros elegidos | MAE CV | MAE validación | RMSE validación | Tiempo |
|---|---|---|---|---|---|
| GridSearchCV | max_samples=0.8, n_estimators=10 | 9,687 | 9,538 | 14,370 | 745 s |
| RandomizedSearchCV | max_features=0.934, max_samples=0.698, n_estimators=6 | 9,926 | 9,795 | 14,467 | 606 s |

Regresión · GridSearchCV: configuraciones probadas

| Hiperparámetros | MAE CV | RMSE CV | Seg. por ajuste |
|---|---|---|---|
| max_samples=0.5, n_estimators=5 | 9,8379 | 14,7387 | 31,8 |
| max_samples=0.5, n_estimators=10 | 9,8127 | 14,7050 | 63,4 |
| max_samples=0.8, n_estimators=5 | 9,7008 | 14,6850 | 41,5 |
| max_samples=0.8, n_estimators=10 | 9,6868 | 14,6500 | 75,3 |

Regresión · RandomizedSearchCV: configuraciones probadas

| Hiperparámetros | MAE CV | RMSE CV | Seg. por ajuste |
|---|---|---|---|
| max_features=0.812, max_samples=0.875, n_estimators=7 | 10,2167 | 14,8442 | 47,9 |
| max_features=0.934, max_samples=0.698, n_estimators=6 | 9,9261 | 14,7100 | 43,3 |
| max_features=0.747, max_samples=0.429, n_estimators=12 | 10,7329 | 15,1237 | 53,0 |
| max_features=0.8, max_samples=0.471, n_estimators=7 | 10,3468 | 14,8972 | 34,6 |

### Clasificación — mejor configuración de cada búsqueda

| Método | Hiperparámetros elegidos | AUC CV | AUC validación | F1_macro validación | Tiempo |
|---|---|---|---|---|---|
| GridSearchCV | max_samples=0.8, n_estimators=10 | 0,9057 | 0,9074 | 0,7121 | 1.179 s |
| RandomizedSearchCV | max_features=0.934, max_samples=0.698, n_estimators=6 | 0,9051 | 0,9070 | 0,7033 | 942 s |

Clasificación · GridSearchCV: configuraciones probadas

| Hiperparámetros | AUC CV | F1_macro CV | Seg. por ajuste |
|---|---|---|---|
| max_samples=0.5, n_estimators=5 | 0,9034 | 0,6970 | 51,9 |
| max_samples=0.5, n_estimators=10 | 0,9045 | 0,6973 | 102,9 |
| max_samples=0.8, n_estimators=5 | 0,9051 | 0,7009 | 55,0 |
| max_samples=0.8, n_estimators=10 | 0,9057 | 0,7013 | 109,9 |

Clasificación · RandomizedSearchCV: configuraciones probadas

| Hiperparámetros | AUC CV | F1_macro CV | Seg. por ajuste |
|---|---|---|---|
| max_features=0.812, max_samples=0.875, n_estimators=7 | 0,9035 | 0,6900 | 63,9 |
| max_features=0.934, max_samples=0.698, n_estimators=6 | 0,9051 | 0,6947 | 59,2 |
| max_features=0.747, max_samples=0.429, n_estimators=12 | 0,9015 | 0,6789 | 85,8 |
| max_features=0.8, max_samples=0.471, n_estimators=7 | 0,9018 | 0,6866 | 56,6 |

### Interpretación

- **Bagging no mejora en ninguna tarea.** En regresión, el mejor Bagging obtiene un MAE de validación de 9,54, frente a 9,40 del boosting solo. En clasificación, su AUC es 0,9074, frente a 0,9075 de la EP2 y 0,9083 del mejor boosting.
- **Cuesta entre 4 y 6 veces más:** entre 606 y 1.179 segundos por búsqueda.
- **Por qué:** Bagging reduce la varianza, y HistGradientBoosting, con parada temprana y regularización, ya tiene poca (su AUC varía apenas 0,001 entre pliegues). En cambio, cada copia ve solo entre el 50% y el 80% de las canciones, lo que aumenta el sesgo. Con max_features menor que 1, algunas copias pierden variables clave como la del artista: la configuración que usó el 75% de las variables fue la peor (MAE CV 10,73).
- Las configuraciones mejoran a medida que cada copia ve más datos y hay más copias, es decir, a medida que se parecen al modelo único.

# 10. Comparación de todas las combinaciones y elección

Se ordenan las 9 combinaciones de cada tarea por su métrica principal en **validación**, la vara común. La columna "CV" muestra lo que estimó la validación cruzada de cada combinación. Sirve como referencia, pero no se usa para elegir, porque KFold y StratifiedKFold tienen fuga. La fila verde es la ganadora.

### Regresión — ranking por MAE en validación

| # | Estrategia | Método | Esquema | Hiperparámetros elegidos | MAE CV | MAE val. | RMSE val. |
|---|---|---|---|---|---|---|---|
| 1 | Boosting | CrossValidation | GroupKFold | l2_regularization=0.5, learning_rate=0.03, max_iter=600, max_leaf_nodes=63, min_samples_leaf=20 | 9,588 | 9,404 | 14,377 |
| 2 | Boosting | GridSearchCV | GroupKFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 9,588 | 9,404 | 14,377 |
| 3 | Boosting | GridSearchCV | KFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 9,148 | 9,404 | 14,377 |
| 4 | Boosting | GridSearchCV | StratifiedKFold | learning_rate=0.06, max_leaf_nodes=63, min_samples_leaf=20 | 9,165 | 9,399 | 14,435 |
| 5 | Boosting | RandomizedSearchCV | GroupKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,619 | 9,416 | 14,392 |
| 6 | Boosting | RandomizedSearchCV | StratifiedKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,210 | 9,416 | 14,392 |
| 7 | Boosting | RandomizedSearchCV | KFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 9,219 | 9,416 | 14,392 |
| 8 | Bagging | GridSearchCV | GroupKFold | max_samples=0.8, n_estimators=10 | 9,687 | 9,538 | 14,370 |
| 9 | Bagging | RandomizedSearchCV | GroupKFold | max_features=0.934, max_samples=0.698, n_estimators=6 | 9,926 | 9,795 | 14,467 |

### Clasificación — ranking por AUC en validación

| # | Estrategia | Método | Esquema | Hiperparámetros elegidos | AUC CV | AUC val. | F1_macro val. |
|---|---|---|---|---|---|---|---|
| 1 | Boosting | RandomizedSearchCV | GroupKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9057 | 0,9083 | 0,7088 |
| 2 | Boosting | RandomizedSearchCV | StratifiedKFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9084 | 0,9083 | 0,7088 |
| 3 | Boosting | RandomizedSearchCV | KFold | l2_regularization=0.976, learning_rate=0.0224, max_leaf_nodes=102, min_samples_leaf=39 | 0,9094 | 0,9083 | 0,7088 |

| # | Estrategia | Método | Esquema | Hiperparámetros elegidos | AUC CV | AUC val. | F1_macro val. |
|---|---|---|---|---|---|---|---|
| 4 | Boosting | GridSearchCV | GroupKFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9062 | 0,9081 | 0,7084 |
| 5 | Boosting | GridSearchCV | StratifiedKFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9085 | 0,9081 | 0,7084 |
| 6 | Boosting | GridSearchCV | KFold | learning_rate=0.03, max_leaf_nodes=63, min_samples_leaf=20 | 0,9096 | 0,9081 | 0,7084 |
| 7 | Bagging | GridSearchCV | GroupKFold | max_samples=0.8, n_estimators=10 | 0,9057 | 0,9074 | 0,7121 |
| 8 | Boosting | CrossValidation | GroupKFold | l2_regularization=0, learning_rate=0.03, max_iter=300, max_leaf_nodes=63, min_samples_leaf=50 | 0,9055 | 0,9075 | 0,7068 |
| 9 | Bagging | RandomizedSearchCV | GroupKFold | max_features=0.934, max_samples=0.698, n_estimators=6 | 0,9051 | 0,9070 | 0,7033 |

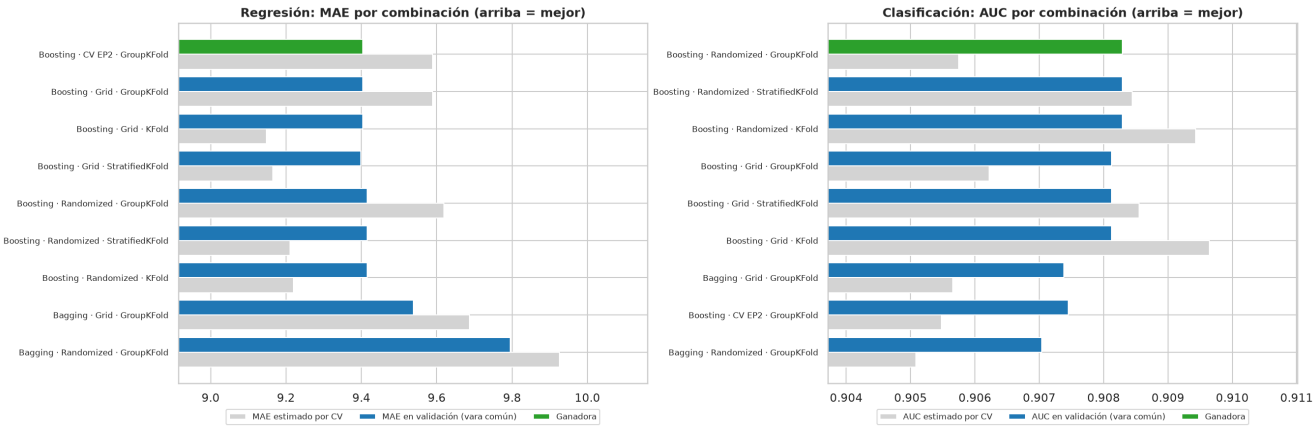



Figura 3. Métrica estimada por validación cruzada (gris) y medida en validación (azul) para cada combinación. La ganadora, en verde.

### Elección

- **Regresión → se mantiene la configuración de la EP2.** Empata en MAE redondeado (9,40) con la grilla bajo KFold y GroupKFold, que eligieron exactamente los mismos valores, y con la grilla bajo StratifiedKFold (learning_rate 0,06), que tiene peor RMSE. Ante igual resultado, se prefiere la opción más simple y barata: 25 s frente a ~170 s.
- **Clasificación → Boosting + RandomizedSearchCV + GroupKFold:** learning_rate 0,0224, max_leaf_nodes 102, min_samples_leaf 39, l2_regularization 0,976 y max_iter 300 (con parada temprana). AUC de validación: 0,9083, frente a 0,9075 de la EP2. KFold y StratifiedKFold eligieron la misma configuración; el desempate favorece al esquema sin fuga.
- La mejora es de +0,0008 de AUC, del mismo orden que la variación entre pliegues (0,0008–0,0018). Por eso se confirma en la prueba final.

# 11. Evaluación final en prueba

La combinación ganadora de cada tarea se reentrena con las 71.740 canciones de entrenamiento y se evalúa **una sola vez** en las 17.843 canciones de prueba de la EP2. El modelo de la EP2 se mide con el mismo código, cargándolo desde `models/`.

En clasificación, el umbral de la clase "alto" se recalcula para el modelo nuevo con la misma regla de la EP2: el mayor umbral u que logra un recall de "alto" de al menos 0,60, usando probabilidades fuera de pliegue del entrenamiento (GroupKFold de 3 pliegues). La regla de decisión es: "alto" si P(alto) ≥ u; si no, la clase más probable entre "bajo" y "medio". El conjunto de prueba no participa en esa elección.

| Regresión | MAE | RMSE | $R^2$ |
|---|---|---|---|
| EP2 (models/) | 9,238 | 14,205 | 0,521 |
| EP3 ganador | 9,238 | 14,205 | 0,521 |

| Clasificación · regla por defecto | ROC-AUC | F1 macro | Accuracy | Recall alto | Precisión alto |
|---|---|---|---|---|---|
| EP2 (models/) | 0,9100 | 0,713 | 0,790 | 0,409 | 0,648 |
| EP3 ganador | 0,9106 | 0,712 | 0,790 | 0,408 | 0,639 |

| Clasificación · umbral de "alto" | Umbral | Recall alto | Precisión alto | F1 macro |
|---|---|---|---|---|
| EP2 (models/) | 0,28 | 0,608 | 0,501 | 0,719 |
| EP3 ganador | 0,28 | 0,614 | 0,513 | 0,724 |

- **Regresión:** el resultado es idéntico al de la EP2, porque es la misma configuración entrenada con los mismos datos.
- **Clasificación:** el AUC sube de 0,9100 a 0,9106. El umbral recalculado vuelve a ser 0,28. Con ese umbral, el recall de "alto" pasa de 0,608 a 0,614, la precisión de "alto" de 0,501 a 0,513 y el F1 macro de 0,719 a 0,724. Con la regla por defecto, el F1 macro queda prácticamente igual (0,712 frente a 0,713).
- La mejora es pequeña, pero consistente entre validación y prueba, y todos los KPIs de la EP2 se siguen cumpliendo. Se adopta el clasificador EP3 (`models/clasificador_popularidad_ep3.joblib` y `models/umbral_clase_alto_ep3.joblib`). Los modelos de la EP2 se conservan para mantener la trazabilidad.

# 12. Conclusiones y limitaciones

| Dimensión | Elección | Motivo |
|---|---|---|
| Esquema de validación | GroupKFold | Sin fuga por reediciones. KFold y StratifiedKFold subestiman el error; estratificar no aporta con tantos datos; LOO es inviable y no estima mejor. |
| Método de búsqueda | RandomizedSearchCV (clasificación) · sin búsqueda (regresión) | Con el mismo presupuesto, la búsqueda aleatoria exploró más hiperparámetros. En regresión, la grilla confirmó los valores de la EP2. |
| Estrategia de ensamble | Boosting | Bagging iguala o empeora y cuesta entre 4 y 6 veces más, porque el modelo ya tiene poca varianza. |

### Limitaciones

- El presupuesto de búsqueda es acotado: 8 configuraciones de boosting, 4 de bagging y 3 pliegues. Una búsqueda más amplia podría encontrar algo mejor, pero las configuraciones

probadas quedan muy cerca entre sí (por ejemplo, el MAE de las 8 configuraciones de la grilla va de 9,59 a 9,72), lo que sugiere poco margen.

- Las mejoras observadas son del orden de la variación entre pliegues. El modelo parece estar cerca de su techo con estas variables: una mejora relevante requeriría información nueva (por ejemplo, la fecha de lanzamiento o la historia de popularidad del artista), más que afinar hiperparámetros.
- La regla de selección redondea el AUC a 3 decimales, como en la EP2. Con esa regla, la diferencia entre el modelo EP3 (0,908) y el EP2 (0,907) es de una unidad en el tercer decimal.

# 13. Guía del notebook y archivos generados

| Sección del notebook | Qué hace | Salidas |
|---|---|---|
| 1. Diseño experimental | Explica los métodos, los esquemas, la vara común, la regla de selección y las combinaciones. | — |
| 2. Preparación | Carga la base y la partición de la EP2 y verifica que coincidan; carga los modelos EP2; define el preprocesamiento, la división en ajuste y validación, las tareas, las métricas y las funciones comunes. | — |
| 3. Bloque A | cross_validate del modelo EP2 en 4 esquemas; fuga y optimismo. | ep3_bloque_A_esquemas_cv.csv · 30_ep3_esquemas_cv.png |
| 4. Bloque B | LOO, KFold 5, KFold 10 y GroupKFold 5 en 500 canciones. | ep3_bloque_B_loo.csv · 31_ep3_leave_one_out.png |
| 5. Bloque C | GridSearchCV y RandomizedSearchCV × 3 esquemas (Boosting). | Tablas de hiperparámetros elegidos y de todas las configuraciones. |
| 6. Bloque D | Bagging sobre el modelo EP2, con Grid y Random (GroupKFold). | Tablas de Bagging. |
| 7. Comparación | Ranking de las 9 combinaciones por tarea y elección de la ganadora. | ep3_comparacion_combinaciones.csv · 32_ep3_comparacion_combinaciones.png |
| 8. Prueba final | Reentrena la ganadora, recalcula el umbral y compara con la EP2 en prueba; guarda los modelos. | ep3_resultados_prueba.csv · clasificador_popularidad_ep3.joblib · umbral_clase_alto_ep3.joblib |
| 9. Conclusiones | Tabla de hiperparámetros por combinación y decisiones finales. | — |

**Cómo ejecutarlo**

- Requiere haber ejecutado antes el notebook EP2, que deja `spotify_multigenero.csv`, `particion_modelo.csv` y los modelos en `models/`. Todos ya están en el repositorio.
- Abrir `notebooks/EP3_Optimizacion_Modelos.ipynb` y ejecutar "Restart & Run All".
- Con `USAR_CACHE = True` (valor por defecto) reutiliza los resultados guardados y termina en cerca de 1 minuto. Para recalcular todo, hay que borrar `data/processed/ep3_cache/` o usar `USAR_CACHE = False`: son unas 2,5 horas en 4 núcleos.

Uso del clasificador nuevo:

```
import joblib
modelo = joblib.load('models/clasificador_popularidad_ep3.joblib')
umbral = joblib.load('models/umbral_clase_alto_ep3.joblib')['umbral_alto'] # 0,28
proba = modelo.predict_proba(X_nuevo) # mismas columnas que en la EP2
# 'alto' si P(alto) >= umbral; si no, la más probable entre 'bajo' y 'medio'
```

# 14. Preguntas frecuentes para la defensa

**¿Por qué no se eligió la combinación con mejor puntaje de validación cruzada?**

Porque los puntajes de distintos esquemas no son comparables: KFold y StratifiedKFold tienen un 12% de fuga y se ven mejor sin serlo. Por eso todas las combinaciones se midieron en la misma validación independiente.

**¿Por qué no se usó LeaveOneOut en la búsqueda?**

Porque serían 57.362 ajustes por configuración, unos 6,6 días cada una. Además, en la submuestra de 500 canciones no estimó mejor que KFold de 10 pliegues y costó unas 70 veces más.

**¿Qué significa estratificar en regresión?**

Se agrupa la popularidad en deciles y cada pliegue conserva la proporción de cada decil. En este dataset no cambió la estimación.

**¿Por qué Bagging no ayudó, si suele mejorar los modelos?**

Bagging reduce la varianza, y el boosting con parada temprana y regularización ya tiene poca. Cada copia entrena con menos canciones, así que aumenta el sesgo, y el promedio no compensa esa pérdida.

**¿Por qué la búsqueda aleatoria ganó en clasificación y no en regresión?**

Con el mismo presupuesto, la búsqueda aleatoria exploró l2_regularization y tamaños de árbol que la grilla dejaba fijos, y en clasificación esa combinación fue algo mejor. En regresión, los valores de la EP2, que la grilla incluía, ya eran los mejores.

**¿La mejora en clasificación es significativa?**

Es pequeña: +0,0008 de AUC en validación y +0,0006 en prueba, del orden de la variación entre pliegues. Lo que la respalda es que va en la misma dirección en validación y en prueba, y que también mejora la precisión y el F1 con el umbral.

**¿Se tocó el conjunto de prueba durante la búsqueda?**

No. Las búsquedas usan solo el conjunto de ajuste, la elección usa la validación y la prueba se usa una sola vez al final, con la ganadora y el modelo EP2.